Day 156: Content-Based Filtering: Recommending from What the Films Are

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 3 | Complete study guide

Watch: Content-Based Recommendations - Recommender Systems Part 1 (DataMListic) | Content-based Filtering (Mahesh Huddar)

Script: `day156_content_based_filtering.py` · every number in this guide is copied by program from `day156_run_output.txt`, the real output of that script.

Days 154 and 155 learned only from who rated what, so a film nobody has rated has no neighbours and no factors. Today each film is described by what it is: MovieLens's 19 genre flags and its decade of release. Each user gets a profile, a ridge regression saying how many stars above their own average each genre and decade is worth to them. On Day 154's split, profiles alone scored 0.9901 on test against 1.0384 for the user's average, and a control with the content shuffled among films scored 1.0418. Added to the bias model they gave 0.9222, still behind Day 154's item-based neighbours (0.9153). Content's real use is the cold start: on films held out entirely, profiles scored 1.0222 where every ratings-only method falls back to 1.0567.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

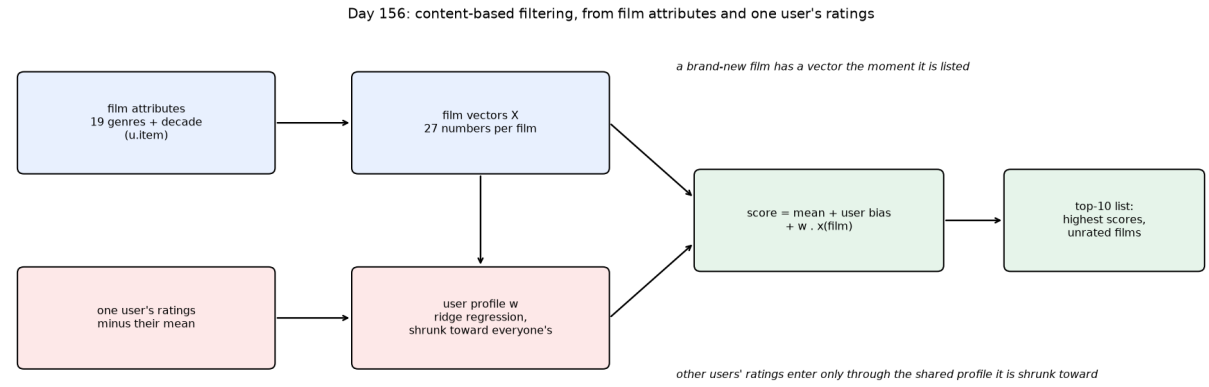
Content-based filtering recommends items similar in their *attributes* to what a user liked before. It needs two things: a description of every item (a **feature vector**, here 27 yes/no flags) and a **user profile**, a weight per feature learned from that user's ratings. A film's score is the profile times the film's features, added up.

Cold start is the problem of a new item (or user) with no ratings. A ratings-only method has nothing to go on; a content-based one has the item's description from the moment it is listed.

REAL-WORLD SCENARIO

A bookshop assistant who knows you love detective novels set in the 1950s can recommend a brand-new one on the day it arrives, before anyone has read it. An assistant who only knows 'people who bought what you bought also bought...' has to wait for sales.

SYSTEM DIAGRAM



Content-based filtering: film vectors from attributes, a profile from one user's ratings, and a score for any film.

Today's part for the week's build

`day156_exercise.py` is today's parts (release years, the feature builder, content similarity, the shared profile, per-user profiles, the prediction and the cold split), checked by hand. Day 154's loaders are imported from `day154_exercise.py`.

ACTUAL OUTPUT

```
Star Wars (1977): load_years gives 1977; films with no year: 1 of 1682
film_features, Action + Sci-Fi from 1977: flags at [1, 15, 24] (1, 15 and 19 + 5 = 24); Drama,
    year missing: [8] (8 only)
cosine of (1, 1, 0) and (1, 0, 1) = 1 / (sqrt 2 x sqrt 2) = 0.5; content_similarity gives 0.5000,
    diagonal 0.0
shared profile, film A residuals 1 and 3, film B residual -2, penalty 2: A (1 + 3) / (2 + 2) =
    1.0, B -2 / (1 + 2) = -0.6667; gives [1.0, -0.6667]
user 2's profile (rated A only, 3), prior 1.0, penalty 2: A 1.0 + (3 - 1.0) x 1 / (1 + 2) =
    1.6667, B stays at the prior -0.6667; gives [1.6667, -0.6667]
prediction for user 2 on film B: profile . features = -0.6667; content_predict gives -0.6667
cold_split of 5 films, 40%: films [0, 2] held out; their ratings at [0, 2, 5, 7]; every one of
    those ratings is a held-out film: True; none missed: True
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Film content: features, content similarity, and how far it agrees with ratings

DEFINITION — start here, no prior knowledge assumed

A **feature vector** lists an item's attributes as numbers. Each film here has 19 genre flags (1 if the film has that genre) and 8 decade flags (exactly one is 1): 27 numbers.

Content similarity is the cosine of two films' vectors: 1 if they share every attribute, 0 if they share none.

REAL-WORLD SCENARIO

Two library books with the same subject headings sit on the same shelf, whether or not anyone has borrowed them.

MATHEMATICS

$$x_i = [\ genre_1, ..., genre_{19}, decade_{<1930}, ..., decade_{1990s}\ ] \in \{0, 1\}^{27}$$

$$cos(a, b) = x_a \cdot x_b \,/\, (|x_a|\ |x_b|)$$

IMPLEMENTATION

```
def film_features(genres, years):
    dec = np.clip((years - 1920) // 10, 0, 7)
    D = np.eye(8)[dec] * (years > 0)[:, None]
    return np.c_[genres, D].astype(float)
def content_similarity(X):
    Xn = X / np.maximum(np.linalg.norm(X, axis=1, keepdims=True), 1e-12)
    S = Xn @ Xn.T
    np.fill_diagonal(S, 0.0)
    return S
```

SPECIFICATION

Content	19 genre flags from u.item; the year from the release date (one film has none, so no decade flag)
Agreement test	for each film with 50+ training ratings: content cosine with its 10 nearest films by Day 154's rating similarity, against 10 random films (seed 0)

ACTUAL OUTPUT

```
1682 films x 27 features (19 genres + 8 decades); genres per film: 833 films with 1, 569 films with 2, 215 films with 3, 51
    films with 4, 11 films with 5, 3 films with 6
release year missing for 1 film(s); years 1922-1998; 1990s films 1348 (80%)
Star Wars (1977): Action, Adventure, Romance, Sci-Fi, War, 1970s
  content neighbours: Return of the Jedi (1983) (0.83); Star Trek: The Motion Picture (1979) (0.82); Empire Strikes Back,
      The (1980) (0.77); Starship Troopers (1997) (0.73); African Queen, The (1951) (0.73)
  (2 films share the fifth one's cosine; ties are broken by file order, not merit)
  rating neighbours (Day 154): Return of the Jedi (1983) (content 0.83); Empire Strikes Back, The (1980) (content 0.77);
      Raiders of the Lost Ark (1981) (content 0.47); Godfather, The (1972) (content 0.41); Usual Suspects, The (1995)
      (content 0.00)
films with 50+ ratings (519): mean content cosine with their 10 rating neighbours 0.443, with 10 random films 0.322; same
    genres and decade (cosine 1): 5.1% against 2.1%
Actual output from this lesson's run.
```

What it means

The content is coarse: 833 films have a single genre, and 1,348 of the 1,682 films are from the 1990s, so the decade flag separates old films far more than recent ones.

Content and ratings agree only partly. Star Wars's closest film by content, Return of the Jedi (cosine 0.83), is also its closest by ratings, but Star Trek: The Motion Picture (0.82) is second by content, and The Usual Suspects, one of its five closest rating neighbours, shares no attribute at all (content 0.00). Across films, rating neighbours have a mean content cosine of 0.443 against 0.322 for random films: content carries some of what ratings capture, far from all of it.

WHY CHOOSE THIS?

Features exist for every film, rated or not, and the similarity is easy to explain ('both are 1970s sci-fi').

WHY NOT THIS? (TRADEOFFS)

Genre and decade are crude; many films have identical vectors (ties broken arbitrarily), and nothing captures quality.

2. User profiles: a ridge regression per user

DEFINITION — start here, no prior knowledge assumed

A **user profile** here is a weight per feature, fitted by **ridge regression** (least squares with a penalty on the weights' size) to the user's ratings minus their own average. A weight of +0.3 on Drama means 'this user rates dramas 0.3 stars above their usual'.

A user with 20 ratings cannot pin down 27 weights, so each profile is **shrunk toward a shared profile** fitted on everyone's ratings: with few ratings, a user looks like the average user.

REAL-WORLD SCENARIO

A new employee's lunch order is guessed from what the whole office likes, and adjusted as you learn their own habits.

MATHEMATICS

$$\textit{shared:}\quad w_0 = \operatorname{argmin} \Sigma_{\textit{known }(u,i)} (t_{ui} - x_i \cdot w)^2 + \lambda_0 |w|^2, \quad t_{ui} = r_{ui} - \mu - b_u$$

$$\textit{user } u:\quad w_u = w_0 + (X_u{}^T X_u + \lambda I)^{-1} X_u{}^T (t_u - X_u w_0); \quad \textit{prediction} = \mu + b_u + x_i \cdot w_u$$

IMPLEMENTATION

```
def fit_shared_profile(X, T, M, lam):
    uu, ii = np.nonzero(M)
    A = X[ii]
    return np.linalg.solve(A.T @ A + lam * np.eye(X.shape[1]), A.T @ T[uu, ii])
def fit_profiles(X, T, M, lam, prior=None):
    w0 = np.zeros(X.shape[1]) if prior is None else prior
    W = np.tile(w0, (M.shape[0], 1))
    for a in range(M.shape[0]):
        j = np.flatnonzero(M[a]); A = X[j]
        W[a] += np.linalg.solve(A.T @ A + lam * np.eye(X.shape[1]), A.T @ (T[a, j] - A @ w0))
    return W
def content_predict(W, X, u, i):
    return (W[u] * X[i]).sum(1)
```

SPECIFICATION

Split	Day 154's: 80,000 training ratings (72,000 + 8,000 for validation), 20,000 test ratings
Content only	mean + user bias (Day 154's penalty 5, no film bias) + profile; the target is what the user's mean leaves
Bias + content	Day 154's bias model + profile; the target is what the bias model leaves
Grid	shared-profile penalty 0.01 to 10,000; profile penalty 0.3 to 1,000, both chosen on validation
Control	the same model with every film given another film's content (seed 0)

ACTUAL OUTPUT

```
validation RMSE (0.4 s): mean + user bias 1.0527; + shared profile (penalty 1) 1.0244; bias model 0.9511
  penalty  content only  bias + content   (per-user profiles, shrunk toward the shared one)
      0.3        1.0755          0.9885
        1        1.0481          0.9659
        3        1.0253          0.9475
       10        1.0100          0.9358
       30        1.0073          0.9349
      100        1.0127          0.9406
      300        1.0185          0.9459
     1000        1.0222          0.9491
chosen on validation: content only, penalty 30; bias + content, penalty 30
                    test, 20,000 ratings    RMSE
                        mean + user bias  1.0384
            content only (user profiles)  0.9901
   control: content shuffled among films  1.0418
                    bias model (Day 154)  0.9401
```

```
           bias model + content profiles  0.9222
              item-based, k 40 (Day 154)  0.9153
test RMSE by how many training ratings the film has:
   ratings  test ratings  content only  bias model  bias + content  item-based
         0            53        0.9550      1.0525          1.0310      1.0525
       1-5           315        1.1160      1.0893          1.0589      1.1165
      6-20          1105        1.1185      1.0376          1.0148      1.0335
     21-50          2948        1.0318      0.9685          0.9492      0.9512
    51-150          8413        0.9744      0.9334          0.9194      0.9078
      151+          7166        0.9639      0.9117          0.8913      0.8779
Actual output from this lesson's run.
```

What it means

Profiles carry real taste: content only scored 0.9901 against 1.0384 for the user's average, and the shuffled-content control scored 1.0418, no better than the average. On top of the bias model, profiles helped (0.9401 to 0.9222) but did not catch Day 154's item-based model (0.9153), which uses far richer information: which particular films a user liked, not only their genres.

By film popularity, bias + content beat the bias model in every bin, most clearly for films with 1–5 training ratings (1.0589 against 1.0893). Item-based neighbours won for well-rated films (0.8779 at 151+). For the 53 test ratings of films with no training ratings, content only scored 0.9550 against 1.0525, but 53 ratings is too few to lean on; section 3 tests that properly.

WHY CHOOSE THIS?

A small, cheap model per user (27 weights); every weight can be read and explained.

WHY NOT THIS? (TRADEOFFS)

It only knows the features it is given; two films with identical genres and decade always get the same score.

3. Cold start: films nobody has rated

DEFINITION — start here, no prior knowledge assumed

To test new films fairly, whole films are held out: every rating of 20% of the films goes to test, every rating of another 10% to validation, and the models see only the remaining 70% of films.

Content kNN is Day 154's item-based predictor with the content similarity in place of the rating similarity: a new film borrows the user's ratings of films that look like it.

REAL-WORLD SCENARIO

Pricing a newly built house: there are no past sales of it, so you look at sales of houses with the same number of rooms in the same area.

IMPLEMENTATION

```
def cold_split(i, ni, frac, seed, exclude=None):
    pool = np.arange(ni) if exclude is None else np.flatnonzero(~exclude)
    rng = np.random.RandomState(seed); cold = np.zeros(ni, bool)
    cold[rng.permutation(pool)[:int(round(frac * ni))]] = True
    return cold, np.flatnonzero(cold[i])
```

ACTUAL OUTPUT

```
films: 1178 for training (68871 ratings), 168 for validation (12182), 336 for test (18947); every rating of a held-out film
    is held out
  a held-out film's bias in Day 154's model is 0.0, and it has no raters, so the bias model, item-based
  neighbours and learned factors all fall back to mean + user bias for it
validation (new films): shared profile penalty 100 (1.0136); profiles penalty 10 (0.9919); content kNN k 20 (1.0099)
  profiles by penalty: 0.3: 1.0614, 1: 1.0310, 3: 1.0061, 10: 0.9919, 30: 0.9920, 100: 0.9998, 300: 1.0067, 1000: 1.0110
  content kNN by k: 5: 1.0516, 10: 1.0220, 20: 1.0099, 40: 1.0117, 80: 1.0174, 160: 1.0205, 320: 1.0213
           test: 18947 ratings of new films    RMSE
 mean + user bias (all ratings-only methods)  1.0567
   shared profile (genre and decade effects)  1.0369
        user profiles (shrunk toward shared)  1.0222
                                content kNN  1.0310
         control: profiles, content shuffled  1.0729
  profiles shrunk toward 0 rather than the shared profile: 1.0221
Actual output from this lesson's run.
```

What it means

For a new film the bias model, item-based neighbours and learned factors all reduce to the user's average, 1.0567. Content improved on it: the shared profile (genre and decade effects alone) 1.0369, content kNN 1.0310, and per-user profiles 1.0222, the best. The control with shuffled content scored 1.0729, worse than no content: wrong descriptions mislead.

Shrinking the profiles toward 0 instead of the shared profile made no difference here (1.0221 against 1.0222). A likely reason, not tested here: at the chosen penalty most users have enough ratings for their own weights to dominate.

WHY CHOOSE THIS?

It is the only method of the week that can rank a film on the day it appears.

WHY NOT THIS? (TRADEOFFS)

The gain is modest (about 0.035 stars of RMSE): genre and decade say little about whether a film is good.

4. Top-10 lists: accuracy and narrowness

DEFINITION — start here, no prior knowledge assumed

A common worry about content-based lists is **overspecialisation**: a user who liked three westerns is shown only westerns. One simple check is the number of distinct genres in each 10-film list.

Precision@10 is Day 154's measure: the share of the 10 recommended films that the user rated 4 or 5 in the test set.

REAL-WORLD SCENARIO

A music service that only ever plays the one band you liked last week.

ACTUAL OUTPUT

```
                     precision@10  median ratings  unrated films  distinct genres
         content only        0.0353              76           6.6%              9.2
  bias model + content        0.0627             100           0.0%              8.2
           bias model        0.0439              99           0.0%              8.2
           popularity        0.1381             346           0.0%             10.7
               random        0.0084              19           2.8%              8.6
Actual output from this lesson's run.
```

What it means

Content alone made poor lists (0.0353), below the bias model (0.0439). Adding profiles to the bias model gave the best personalised lists so far (0.0627), though recommending the most-rated films still scored far higher (0.1381). Content-only lists did include films with no training ratings (6.6%), which no ratings-only list can.

The expected narrowness did not appear on this measure: content-only lists covered 9.2 distinct genres on average against 8.2 for the bias model. Genre counts are a coarse check, and a profile spreads weight over many genres and decades; a sharper measure of narrowness is left for Day 159.

WHY CHOOSE THIS?

Lists can contain brand-new films, and each choice has a reason that can be shown to the user.

WHY NOT THIS? (TRADEOFFS)

Accuracy on its own is low here; the attributes are too coarse to separate good films from bad ones in a genre.

5. Explaining a recommendation from one user's profile

DEFINITION — start here, no prior knowledge assumed

Because the score is a sum of weights, each recommendation can be explained exactly: the film's features and what each one adds. The user is the first in the file, not chosen for a good story.

REAL-WORLD SCENARIO

A loan decision that comes with 'income +, existing debt –' instead of a bare yes or no.

ACTUAL OUTPUT

```
user 1's content profile (stars above the user's mean): likes Documentary +0.52, Animation +0.38, Drama +0.33, Sci-Fi +0.31
                                  dislikes Children's -0.60, Adventure -0.22, Musical -0.21, 1990s -0.20
  Day the Earth Stood Still, The (1951)  +0.84 = +0.33 (Drama) +0.31 (Sci-Fi) +0.20 (1950s)
  Manhattan (1979)                       +0.83 = +0.33 (Drama) +0.30 (1970s) +0.24 (Romance) -0.04 (Comedy)
  Witness (1985)                         +0.77 = +0.33 (Drama) +0.24 (Romance) +0.17 (1980s) +0.04 (Thriller)
  Casablanca (1942)                      +0.74 = +0.33 (Drama) +0.24 (Romance) +0.17 (1940s) +0.00 (War)
  Somewhere in Time (1980)               +0.74 = +0.33 (Drama) +0.24 (Romance) +0.17 (1980s)
  unrated films sharing the top score: 1
Actual output from this lesson's run.
```

What it means

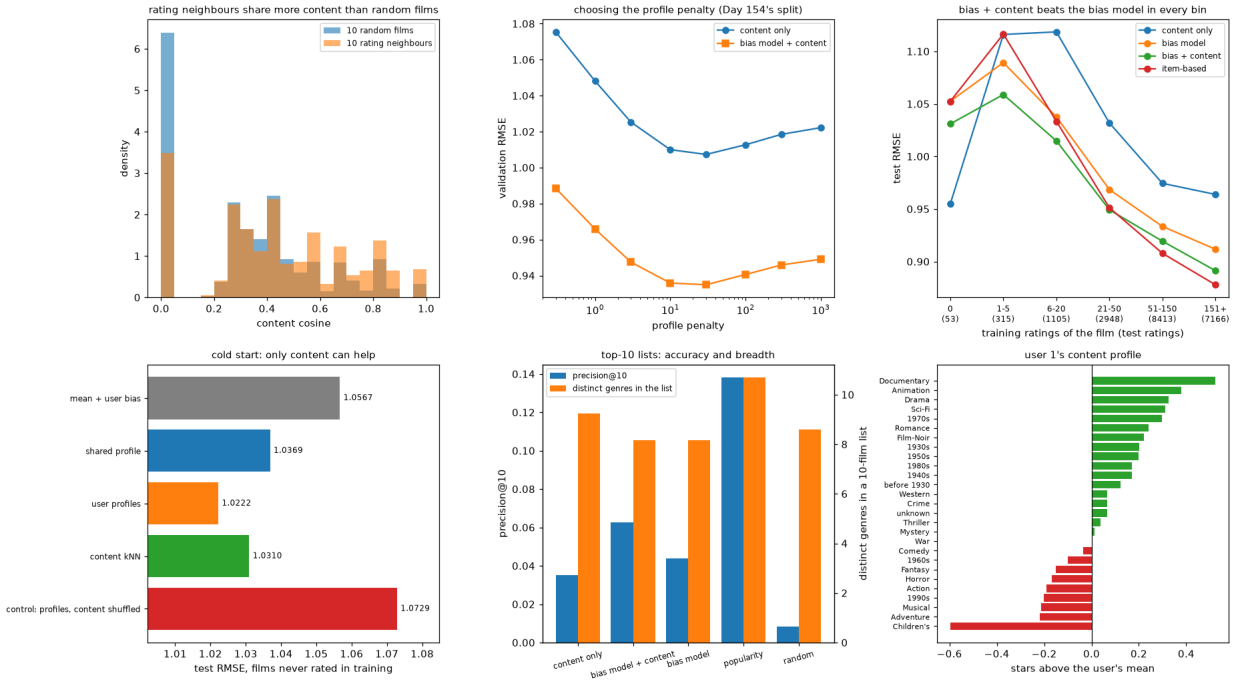
User 1's profile likes documentaries (+0.52 stars) and dramas (+0.33) and dislikes children's films (–0.60). The top recommendation, The Day the Earth Stood Still, scores +0.84: Drama + Sci-Fi + 1950s. The explanation is exact, but it also shows the weakness: Witness, Casablanca and Somewhere in Time are recommended for the same reasons (Drama, Romance, an older decade), however different the films.

WHY CHOOSE THIS?

Transparent: every score is a short, exact sum a user can read.

WHY NOT THIS? (TRADEOFFS)

The explanation is only as rich as the features; here, genre and decade.



Top: content cosine of films with their rating neighbours against random films; choosing the profile penalty; test RMSE by the film's number of training ratings. Bottom: cold-start test RMSE; top-10 precision and genre breadth; user 1's profile.

Interview preparation — technical questions

Q: When would you use content-based filtering over collaborative filtering?

A: For new items with no ratings. Here, on held-out films, profiles scored 1.0222 where ratings-only methods fall back to 1.0567.

Q: How do you build a user profile?

A: Regress the user's ratings (minus their mean) on the items' features, with a ridge penalty; shrink toward a profile fitted on everyone so users with few ratings get sensible weights.

Q: How do you test cold start honestly?

A: Hold out whole items, every rating of them, for test and for validation; a random split of ratings leaves almost every film with some training ratings.

Q: How do you check content features carry signal?

A: Shuffle the features among items and refit: 1.0729 on new films here, worse than no content (1.0567).

Q: Why did content-based lists not beat collaborative ones?

A: Genre and decade separate kinds of films, not good films from bad; precision@10 was 0.0353 against 0.0439 for the bias model.

Interview preparation — theoretical questions

Q: What is overspecialisation?

A: Recommending only items very like past ones. It did not show on a genre count here (9.2 genres per list against 8.2), but the measure is coarse.

Q: Does content-based filtering solve the new-user problem?

A: No: a new user has no ratings, so no profile beyond the shared one. It helps new items.

Q: Why shrink toward a shared profile rather than zero?

A: Zero says 'no taste'; the shared profile says 'average taste'. Here the two gave the same cold-start error (1.0221 and 1.0222).

Q: Is content kNN different from item-based CF?

A: Same predictor (Day 154's), different similarity: attributes instead of co-ratings, so it works for films without raters.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Describing films by their attributes lets a recommender score any film, including one nobody has rated. Per-user profiles over genre and decade carried real taste (a shuffled control lost it) and were the best model for brand-new films, but for films with ratings they added only a little to the bias model and stayed behind item-based neighbours.

- Day 154 split, test RMSE: user average 1.0384, content only 0.9901, control 1.0418, bias + content 0.9222, item-based 0.9153.
- New films: ratings-only 1.0567, shared profile 1.0369, content kNN 1.0310, profiles 1.0222, shuffled control 1.0729.
- Top-10 precision: bias + content 0.0627, content only 0.0353, popularity 0.1381.

Suggested watch order, if starting from zero

- 1. Content-Based Recommendations - Recommender Systems Part 1 (DataMListic) — a short introduction to content-based recommendations.
- 2. Content-based Filtering (Mahesh Huddar) — a lecture on content-based filtering.

Day 157 preview

- **Hybrid recommenders.** Content places new films; ratings-based models are more accurate once films have ratings. Hybrids combine them: blending predictions, switching by how much is known about a film, or putting content features inside the factor model itself.